28.2 向量记忆搜索

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~320k tokens，输出 ~8k tokens（本节）

上一节我们了解了记忆系统的文件布局和分块机制——Markdown 文件被拆分成若干文本块（Chunk），等待被索引。但"索引"具体意味着什么？如何让 Agent 能够根据用户的自然语言问题找到相关记忆？

答案是**向量搜索（Vector Search）**：将文本块转换为高维向量（即嵌入向量），然后通过余弦相似度找到语义上最接近的结果。本节深入分析 OpenClaw 的嵌入向量引擎、多提供者架构、SQLite 存储方案和 sqlite-vec 向量加速。

28.2.1 嵌入向量（Embeddings）引擎

什么是嵌入向量？

> **衍生解释**：嵌入向量（Embedding）是一种将文本映射到高维浮点数数组的技术。例如，句子 "我喜欢 TypeScript" 可能被映射为一个 1536 维的向量 `[0.023, -0.017, 0.041, ...]` 。语义相近的文本在向量空间中距离更近——"我喜欢 TypeScript" 和 "TypeScript 是我偏好的语言" 的向量夹角很小，而与 "今天天气不错" 的向量夹角很大。通过计算向量间的**余弦相似度（Cosine Similarity）**，可以量化两段文本的语义相关程度。

OpenClaw 的嵌入引擎定义在 `src/memory/embeddings.ts` 中，核心接口是 `EmbeddingProvider` ：

```
// src/memory/embeddings.ts

export type EmbeddingProvider = {
  id: string;                                       // 提供者标识："openai" | "gemini" | "voyage"
  model: string;                                    // 模型名称，如 "text-embedding-3-small"
  embedQuery: (text: string) => Promise<number[]>;  // 将单条文本转换为向量
  embedBatch: (texts: string[]) => Promise<number[][]>; // 批量转换
};
```

这是一个经典的**策略模式（Strategy Pattern）**——所有嵌入提供者实现相同接口，系统在运行时决定使用哪个。 `embedQuery` 用于搜索时将用户查询转换为向量， `embedBatch` 用于索引时批量处理文本块。

提供者选择流程

`createEmbeddingProvider()` 是嵌入引擎的入口。它根据配置决定使用哪个提供者，并实现了**自动检测（auto）** 和**回退（fallback）** 两种容错机制：

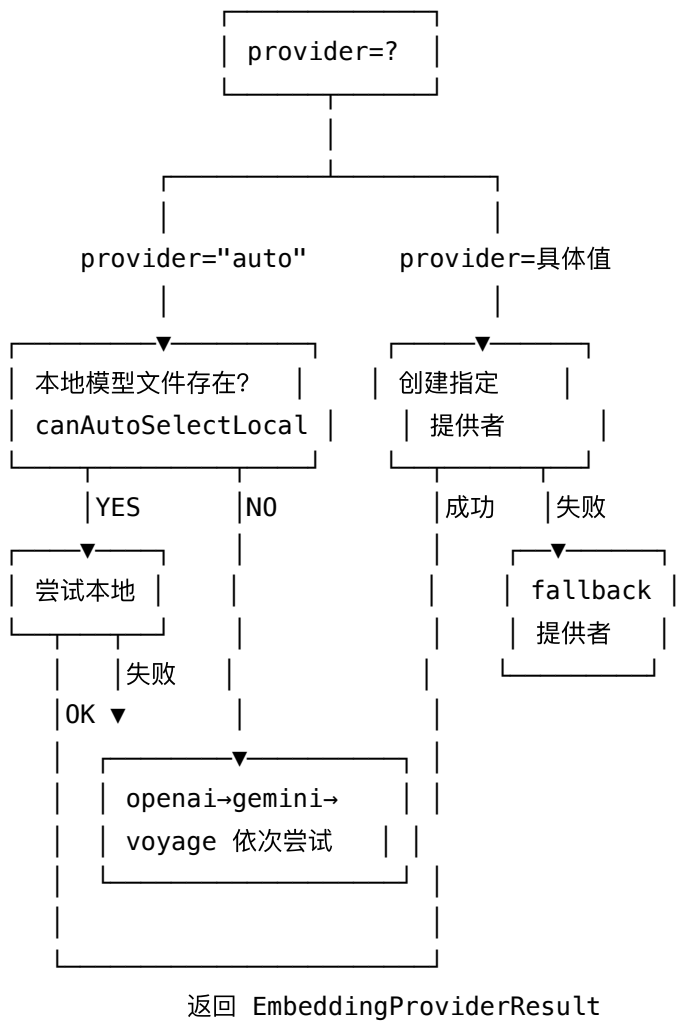
```
// src/memory/embeddings.ts（简化）

export async function createEmbeddingProvider(
  options: EmbeddingProviderOptions,
): Promise<EmbeddingProviderResult> {
  const requestedProvider = options.provider; // "auto" | "openai" | "gemini" | "voyage" | "lo
  const fallback = options.fallback;

  if (requestedProvider === "auto") {
    // 1. 优先尝试本地（如果模型文件存在于磁盘）
    if (canAutoSelectLocal(options)) {
      try { return await createProvider("local"); }
      catch (err) { localError = formatLocalSetupError(err); }
    }
    // 2. 依次尝试远程提供者：openai → gemini → voyage
    for (const provider of ["openai", "gemini", "voyage"] as const) {
      try { return await createProvider(provider); }
      catch (err) {
        if (isMissingApiKeyError(err)) continue; // 没有 API Key，跳过
        throw err;                                // 其他错误，直接抛出
      }
    }
    throw new Error("No embeddings provider available.");
  }

  // 非 auto 模式：直接创建指定提供者，失败时尝试 fallback
  try {
    return await createProvider(requestedProvider);
  } catch (primaryErr) {
    if (fallback && fallback !== "none") {
      return await createProvider(fallback); // 记录 fallbackFrom 和 fallbackReason
    }
    throw primaryErr;
  }
}
```

用流程图表示：



`canAutoSelectLocal` 的判定逻辑值得注意——它**只在本地模型文件确实存在于磁盘时**才返回 `true`，远程 URL 或 Hugging Face Hub 路径（`hf:` 前缀）会被排除：

```
// src/memory/embeddings.ts

function canAutoSelectLocal(options: EmbeddingProviderOptions): boolean {
  const modelPath = options.local?.modelPath?.trim();
  if (!modelPath) return false;
  if (/^(hf:|https?:)/i.test(modelPath)) return false; // 远程路径不自动选择
  const resolved = resolveUserPath(modelPath);
  return fsSync.statSync(resolved).isFile();             // 文件必须存在
}
```

这样避免了 auto 模式下因为下载大模型而阻塞启动。

向量归一化

所有本地生成的嵌入向量在返回前都会经过**归一化（Normalization）**处理：

```
// src/memory/embeddings.ts

function sanitizeAndNormalizeEmbedding(vec: number[]): number[] {
  const sanitized = vec.map(v => Number.isFinite(v) ? v : 0); // 清理 NaN/Infinity
  const magnitude = Math.sqrt(sanitized.reduce((sum, v) => sum + v * v, 0));
  if (magnitude < 1e-10) return sanitized; // 零向量不处理
  return sanitized.map(v => v / magnitude); // L2 归一化
}
```

> **衍生解释**：L2 归一化将向量缩放到单位长度（模为 1）。归一化后，余弦相似度等价于向量点积，计算更高效。此外，归一化确保不同文本块的向量在同一"尺度"上比较，避免因为向量模长差异导致的偏差。

28.2.2 多嵌入提供者支持

OpenClaw 支持四种嵌入提供者，覆盖从云端到本地的完整光谱。

提供者对比

提供者	默认模型	维度	运行位置	API Key	特点
OpenAI	`text-embedding-3-small`	1536	云端	`OPENAI_API_KEY`	最广泛使用，质量稳定
Gemini	`gemini-embedding-001`	768	云端	`GOOGLE_API_KEY`	支持 `taskType` 区分查询/文档
Voyage	`voyage-4-large`	1024	云端	`VOYAGE_API_KEY`	专注检索场景，支持 `input_type`
Local	`embeddinggemma-300M`	取决于模型	本地	无需	完全离线，隐私优先

OpenAI 提供者

OpenAI 的实现最为简洁——标准的 REST API 调用：

```
// src/memory/embeddings-openai.ts（简化）

export const DEFAULT_OPENAI_EMBEDDING_MODEL = "text-embedding-3-small";

async function createOpenAiEmbeddingProvider(options) {
  const client = await resolveOpenAiEmbeddingClient(options);
  const url = `${client.baseUrl}/embeddings`;

  const embed = async (input: string[]): Promise<number[][]> => {
    const res = await fetch(url, {
      method: "POST",
      headers: client.headers, // Authorization: Bearer ${apiKey}
      body: JSON.stringify({ model: client.model, input }),
    });
    const payload = await res.json();
    return payload.data.map(entry => entry.embedding ?? []);
  };

  return {
    provider: {
      id: "openai",
      model: client.model,
      embedQuery: async (text) => (await embed([text]))[0] ?? [],
      embedBatch: embed, // 原生支持批量
    },
    client,
  };
}
```

`embedQuery` 和 `embedBatch` 共用同一个 `embed` 函数——OpenAI 的 `/embeddings` 端点天然支持多文本输入，单条查询只是 `input` 数组长度为 1 的特例。

Gemini 提供者

Gemini 的实现有两个显著不同：

1. **独立的查询/文档端点**：Gemini 提供 `embedContent`（单条）和 `batchEmbedContents`（批量）两个 API，且通过 `taskType` 字段区分用途——查询用 `RETRIEVAL_QUERY`，文档用 `RETRIEVAL_DOCUMENT`。
2. **模型路径格式**：Gemini API 要求 `models/` 前缀（如 `models/gemini-embedding-001`），代码中有专门的 `buildGeminiModelPath` 处理。

```typescript
// src/memory/embeddings-gemini.ts（简化）

const embedUrl = `${baseUrl}/${modelPath}:embedContent`;
const batchUrl = `${baseUrl}/${modelPath}:batchEmbedContents`;

// 查询嵌入
const embedQuery = async (text: string): Promise<number[]> => {
  const res = await fetch(embedUrl, {
    headers: client.headers, // x-goog-api-key: ${apiKey}
    body: JSON.stringify({
      content: { parts: [{ text }] },
      taskType: "RETRIEVAL_QUERY",   // ← 查询类型
    }),
  });
  return (await res.json()).embedding?.values ?? [];
};

// 批量文档嵌入
const embedBatch = async (texts: string[]): Promise<number[][]> => {
  const requests = texts.map(text => ({
    model: modelPath,
    content: { parts: [{ text }] },
    taskType: "RETRIEVAL_DOCUMENT",  // ← 文档类型
  }));
  const res = await fetch(batchUrl, {
    body: JSON.stringify({ requests }),
  });
  return (await res.json()).embeddings.map(e => e.values ?? []);
};
```

> **衍生解释**： `RETRIEVAL_QUERY` 与 `RETRIEVAL_DOCUMENT` 的区别在于嵌入模型内部对文本的处理方式不同。查询通常较短且表达的是"信息需求"，文档较长且包含"信息内容"。这种非对称嵌入（Asymmetric Embedding）能提升检索精度——模型针对查询和文档分别优化向量表示。

Voyage 提供者

Voyage AI 专注于检索场景，其 API 与 OpenAI 类似但通过 `input_type` 字段区分查询和文档：

```
// src/memory/embeddings-voyage.ts（简化）

export const DEFAULT_VOYAGE_EMBEDDING_MODEL = "voyage-4-large";

const embed = async (input: string[], input_type?: "query" | "document") => {
  const body = { model: client.model, input };
  if (input_type) body.input_type = input_type;
  const res = await fetch(url, { body: JSON.stringify(body) });
  return (await res.json()).data.map(entry => entry.embedding ?? []);
};

return {
  provider: {
    id: "voyage",
    model: client.model,
    embedQuery: async (text) => (await embed([text], "query"))[0],   // ← query
    embedBatch: async (texts) => embed(texts, "document"),            // ← document
  },
};
```

本地提供者（node-llama-cpp）

本地提供者是最独特的——它在用户机器上运行一个小型嵌入模型，完全不需要网络。默认使用 Google 的 `embeddinggemma-300M`（GGUF 量化格式，约 300MB）：

```typescript
// src/memory/embeddings.ts（简化）

const DEFAULT_LOCAL_MODEL =
  "hf:ggml-org/embeddinggemma-300M-GGUF/embeddinggemma-300M-Q8_0.gguf";

async function createLocalEmbeddingProvider(options) {
  const modelPath = options.local?.modelPath || DEFAULT_LOCAL_MODEL;
  const { getLlama, resolveModelFile, LlamaLogLevel } = await importNodeLlamaCpp();

  let llama, embeddingModel, embeddingContext; // 懒初始化

  const ensureContext = async () => {
    if (!llama)
      llama = await getLlama({ logLevel: LlamaLogLevel.error });
    if (!embeddingModel) {
      const resolved = await resolveModelFile(modelPath); // 首次自动从 HF Hub 下载
      embeddingModel = await llama.loadModel({ modelPath: resolved });
    }
    if (!embeddingContext)
      embeddingContext = await embeddingModel.createEmbeddingContext();
    return embeddingContext;
  };

  return {
    id: "local",
    model: modelPath,
    embedQuery: async (text) => {
      const ctx = await ensureContext();
      const embedding = await ctx.getEmbeddingFor(text);
      return sanitizeAndNormalizeEmbedding(Array.from(embedding.vector));
    },
    embedBatch: async (texts) => {
      // 逐条处理，每条结果归一化
      return Promise.all(texts.map(async text => {
        const ctx = await ensureContext();
        const embedding = await ctx.getEmbeddingFor(text);
        return sanitizeAndNormalizeEmbedding(Array.from(embedding.vector));
      }));
    },
  };
}
```

几个关键设计决策：

1. **懒加载（Lazy Loading）**： `node-llama-cpp` 是可选依赖，通过动态 `import()` 延迟加载，避免未安装时导致启动失败。

2. **模型文件解析**： `resolveModelFile` 支持本地路径和 `hf:` Hub 链接，首次使用会自动下载到缓存目录。
3. **串行 Batch**：本地模型的 `embedBatch` 实际上是逐条调用 `getEmbeddingFor` ——因为本地推理受限于 CPU/GPU 资源，并行度无法像云端 API 那样扩展。

API Key 解析

所有云端提供者共享统一的 API Key 解析流程。以 OpenAI 为例：

```
// src/memory/embeddings-openai.ts（简化）

const apiKey = remoteApiKey                // 1. 远程配置传入的 Key（如 MCP 服务器）
  ? remoteApiKey
  : requireApiKey(
      await resolveApiKeyForProvider({     // 2. 从全局配置 + 环境变量解析
        provider: "openai",
        cfg: options.config,
        agentDir: options.agentDir,
      }),
      "openai",
    );
```

三层优先级：远程传入 > 配置文件 > 环境变量（ `OPENAI_API_KEY ）。如果都找不到， `requireApiKey` 抛出 "No API key found for provider openai" 异常——在 auto 模式下这个异常会被捕获并跳过该提供者。

28.2.3 SQLite 存储

为什么是 SQLite?

OpenClaw 的记忆索引存储在 SQLite 数据库中。对于一个 CLI 工具来说，SQLite 是理想选择：

- **零服务**：无需启动数据库服务器，单文件即整个数据库
- **进程内**：直接嵌入 Node.js 进程，使用 `node:sqlite` 模块访问（Node.js 22+ 原生支持）
- **事务安全**：ACID 保证，即使进程崩溃也不会损坏索引
- **扩展性**：可以加载 `sqlite-vec` 等扩展实现向量搜索

> **衍生解释**：Node.js 从 22 版本开始内置了 `node:sqlite` 模块（实验性），提供同步 API（ `DatabaseSync` ）直接操作 SQLite 数据库。这意味着不需要像 `better-sqlite3` 这样的第三方 native 模块，减少了安装和兼容性问题。OpenClaw 通过 `requireNodeSqlite()` 包装函数来引入这个内置模块。

Schema 设计

`ensureMemoryIndexSchema()` 函数定义了完整的数据库结构：

```typescript
// src/memory/memory-schema.ts（简化）

export function ensureMemoryIndexSchema(params: {
  db: DatabaseSync;
  embeddingCacheTable: string;  // 默认 "embedding_cache"
  ftsTable: string;             // 默认 "chunks_fts"
  ftsEnabled: boolean;
}): { ftsAvailable: boolean; ftsError?: string } {

  // 1. 元数据表
  params.db.exec(`
    CREATE TABLE IF NOT EXISTS meta (
      key TEXT PRIMARY KEY,
      value TEXT NOT NULL
    );
  `);

  // 2. 文件追踪表
  params.db.exec(`
    CREATE TABLE IF NOT EXISTS files (
      path TEXT PRIMARY KEY,
      source TEXT NOT NULL DEFAULT 'memory',
      hash TEXT NOT NULL,
      mtime INTEGER NOT NULL,
      size INTEGER NOT NULL
    );
  `);

  // 3. 文本块表（核心）
  params.db.exec(`
    CREATE TABLE IF NOT EXISTS chunks (
      id TEXT PRIMARY KEY,
      path TEXT NOT NULL,
      source TEXT NOT NULL DEFAULT 'memory',
      start_line INTEGER NOT NULL,
      end_line INTEGER NOT NULL,
      hash TEXT NOT NULL,
      model TEXT NOT NULL,
      text TEXT NOT NULL,
      embedding TEXT NOT NULL,
      updated_at INTEGER NOT NULL
    );
  `);

  // 4. 嵌入缓存表
  params.db.exec(`
```

```
    CREATE TABLE IF NOT EXISTS ${params.embeddingCacheTable} (
      provider TEXT NOT NULL,
      model TEXT NOT NULL,
      provider_key TEXT NOT NULL,
      hash TEXT NOT NULL,
      embedding TEXT NOT NULL,
      dims INTEGER,
      updated_at INTEGER NOT NULL,
      PRIMARY KEY (provider, model, provider_key, hash)
    );
  `);
  // ...索引创建略
}
```

四张表各司其职：

表名	主键	作用
`meta`	`key`	存储索引元数据（如向量维度 `vectorDims`）
`files`	`path`	追踪已索引的文件及其 hash，用于增量同步
`chunks`	`id`	存储文本块的完整信息（文本 + 嵌入向量 + 位置）
`embedding_cache`	复合键	缓存已计算的嵌入，避免重复调用 API

`chunks` 表是核心——每个文本块对应一行，`embedding` 字段以 JSON 字符串形式存储完整的浮点数组。`model` 字段记录生成嵌入时使用的模型，确保不同模型的向量不会混搭比较。

`files` 表中的 `hash` 字段实现了**增量索引**：同步时比较文件当前 hash 与存储的 hash，只有发生变化的文件才需要重新分块和嵌入。`source` 字段区分记忆来源（`"memory"` 表示记忆文件，`"session"` 表示会话记录）。

FTS5 全文搜索表

Schema 创建函数还会尝试建立 FTS5 虚拟表：

```
// src/memory/memory-schema.ts（简化）

if (params.ftsEnabled) {
  try {
    params.db.exec(
      `CREATE VIRTUAL TABLE IF NOT EXISTS ${params.ftsTable} USING fts5(
        text,                    -- 全文搜索的文本内容
        id UNINDEXED,           -- 以下字段仅存储，不参与全文索引
        path UNINDEXED,
        source UNINDEXED,
        model UNINDEXED,
        start_line UNINDEXED,
        end_line UNINDEXED
      );`
    );
    ftsAvailable = true;
  } catch (err) {
    ftsAvailable = false;       // FTS5 不可用时优雅降级
    ftsError = err.message;
  }
}
```

> **衍生解释**：FTS5（Full-Text Search 5）是 SQLite 内置的全文搜索引擎。它为文本建立倒排索引（Inverted Index），支持 `MATCH` 查询和 BM25 排名。 `UNINDEXED` 关键字表示该列只存储数据但不建立全文索引——这里只需要对 `text` 列做全文搜索，其他列用于关联查询。

注意 FTS5 的创建被 `try/catch` 包裹——某些 SQLite 编译版本可能不包含 FTS5 支持，此时系统会优雅降级为纯向量搜索。

28.2.4 sqlite-vec 向量加速

为什么需要向量加速?

`chunks` 表已经存储了嵌入向量（作为 JSON 字符串）。最简单的搜索方式是：读取所有 chunks，解析每个向量，逐一计算余弦相似度，然后排序——这就是 `listChunks` + `cosineSimilarity` 的暴力搜索路径。但当记忆文件很多时，这个 O(n) 的逐一计算会变慢。

`sqlite-vec` 是一个 SQLite 扩展，通过向量虚拟表提供**原生的向量距离计算**，直接在 SQL 查询中执行 `vec_distance_cosine()` ，性能远超逐行 JavaScript 计算。

扩展加载

```
// src/memory/sqlite-vec.ts

export async function loadSqliteVecExtension(params: {
  db: DatabaseSync;
  extensionPath?: string;
}): Promise<{ ok: boolean; extensionPath?: string; error?: string }> {
  try {
    const sqliteVec = await import("sqlite-vec");
    const extensionPath = params.extensionPath ?? sqliteVec.getLoadablePath();
    params.db.enableLoadExtension(true);  // 必须先启用扩展加载
    sqliteVec.load(params.db);
    return { ok: true, extensionPath };
  } catch (err) {
    return { ok: false, error: err.message }; // 加载失败不是致命错误
  }
}
```

`sqlite-vec` 通过 npm 包分发， `getLoadablePath()` 返回编译好的 `.dylib/.so/.dll` 文件路径。加载失败只记录错误，不会阻止系统运行——系统会回退到暴力搜索。

向量表管理

加载扩展后， `MemoryIndexManager` 会创建一个 `vec0` 虚拟表：

```
// src/memory/manager.ts（简化）

private async ensureVectorReady(dimensions?: number): Promise<boolean> {
  if (!this.vector.enabled) return false;

  // 懒加载 sqlite-vec 扩展（带 30s 超时）
  if (!this.vectorReady) {
    this.vectorReady = this.withTimeout(
      this.loadVectorExtension(),
      VECTOR_LOAD_TIMEOUT_MS,   // 30_000ms
      `sqlite-vec load timed out after 30s`,
    );
  }

  const ready = await this.vectorReady;
  if (ready && dimensions) {
    this.ensureVectorTable(dimensions); // 确保向量表维度匹配
  }
  return ready;
}

private ensureVectorTable(dimensions: number): void {
  if (this.vector.dims === dimensions) return;   // 维度未变，无需操作
  if (this.vector.dims && this.vector.dims !== dimensions) {
    this.dropVectorTable();                       // 维度变化，先删再建
  }
  this.db.exec(
    `CREATE VIRTUAL TABLE IF NOT EXISTS chunks_vec USING vec0(
      id TEXT PRIMARY KEY,
      embedding FLOAT[${dimensions}]
    )`
  );
  this.vector.dims = dimensions;
}
```

`vec0` 是 sqlite-vec 提供的虚拟表类型。`FLOAT[${dimensions}]` 声明了固定维度的浮点向量列。如果用户切换了嵌入模型（维度改变），向量表会被**完全重建**——因为不同维度的向量无法比较。

向量搜索实现

有了 `chunks_vec` 表，搜索就可以利用 SQL 来完成：

```typescript
// src/memory/manager-search.ts

const vectorToBlob = (embedding: number[]): Buffer =>
  Buffer.from(new Float32Array(embedding).buffer);

export async function searchVector(params: {
  db: DatabaseSync;
  vectorTable: string;       // "chunks_vec"
  providerModel: string;     // 确保只比较同模型的向量
  queryVec: number[];
  limit: number;
  snippetMaxChars: number;
  ensureVectorReady: (dimensions: number) => Promise<boolean>;
  sourceFilterVec: { sql: string; params: string[] };
  sourceFilterChunks: { sql: string; params: string[] };
}): Promise<SearchRowResult[]> {
  if (params.queryVec.length === 0) return [];

  // 路径 A: sqlite-vec 可用 → SQL 向量搜索
  if (await params.ensureVectorReady(params.queryVec.length)) {
    const rows = params.db.prepare(
      `SELECT c.id, c.path, c.start_line, c.end_line, c.text, c.source,
              vec_distance_cosine(v.embedding, ?) AS dist
         FROM chunks_vec v
         JOIN chunks c ON c.id = v.id
        WHERE c.model = ?
        ORDER BY dist ASC
        LIMIT ?`
    ).all(
      vectorToBlob(params.queryVec),  // 查询向量转为二进制 Blob
      params.providerModel,
      params.limit
    );
    return rows.map(row => ({
      ...row,
      score: 1 - row.dist,            // 距离转相似度: distance = 1 - cosine_similarity
    }));
  }

  // 路径 B: sqlite-vec 不可用 → JavaScript 暴力搜索
  const candidates = listChunks({ db, providerModel, sourceFilter });
  const scored = candidates
    .map(chunk => ({ chunk, score: cosineSimilarity(queryVec, chunk.embedding) }))
    .filter(entry => Number.isFinite(entry.score));
  return scored.toSorted((a, b) => b.score - a.score).slice(0, limit);
}
```

两条路径的性能差异：

方面	sqlite-vec（路径 A）	暴力搜索（路径 B）
向量计算	C 实现，在 SQLite 引擎内	JavaScript `cosineSimilarity()`
数据传输	只传输 Top-K 结果	所有 chunks 的嵌入向量都要从 DB 读入 JS
排序	SQL `ORDER BY`	JavaScript `.toSorted()`
内存占用	低（流式处理）	高（所有向量加载到内存）

注意 `vectorToBlob` 函数——sqlite-vec 要求向量以 `Float32Array` 的二进制 Buffer 形式传入，而非 JSON 数组。这避免了序列化/反序列化开销。

双路径降级策略

整个向量搜索的降级策略形成了一个清晰的层次：

```
sqlite-vec 加载成功?
    ├── YES → vec0 虚拟表 + vec_distance_cosine()（快速）
    └── NO  → 从 chunks 表读取 embedding JSON
              → parseEmbedding() 解析
              → cosineSimilarity() 逐一计算（慢但正确）
```

这种设计确保了**功能的一致性**——无论 sqlite-vec 是否可用，用户都能搜索记忆，只是速度不同。对于记忆文件不多的个人用户（几十到几百个 chunks），暴力搜索的延迟通常也可以接受。

本节小结

1. **嵌入向量引擎**基于策略模式， `EmbeddingProvider` 接口统一了 `embedQuery` 和 `embedBatch` 两个操作，所有提供者实现相同契约。
2. **四种提供者**覆盖不同场景：OpenAI（通用）、Gemini（非对称嵌入）、Voyage（检索优化）、Local（离线隐私）。 `auto` 模式按 local → openai → gemini → voyage 顺序自动选择。
3. **SQLite 存储**采用四张表： `meta` （元数据）、 `files` （文件追踪）、 `chunks` （文本块 + 嵌入）、 `embedding_cache` （嵌入缓存），通过 `hash` 字段实现增量索引。
4. **FTS5 虚拟表**为全文搜索提供 BM25 排名能力，不可用时优雅降级。
5. **sqlite-vec 扩展**提供原生向量距离计算，通过 `vec0` 虚拟表和 `vec_distance_cosine()` 函数实现 SQL 内向量搜索。不可用时自动回退到 JavaScript 暴力搜索。
6. **双路径降级**贯穿整个设计——从嵌入提供者的 auto/fallback，到向量搜索的 sqlite-vec/暴力计算，每个环节都有降级方案，确保功能永远可用。